Architectural Alignment Summary

- **Java & Dependency Versions:** Java 21 release targeting, Selenium 4.46.0, TestNG 7.12.0, and OpenCSV 5.12.0.
- **Driver Management & Isolation:** Selenium Manager (native driver discovery) managed through a thread-isolated ThreadLocal<WebDriver>.
- **Page Object Design:** Direct By locators encapsulated within Page Objects (avoiding @FindBy Page Factory annotations to prevent stale element issues) coupled with explicit WaitUtils.
- **Configuration:** Validated startup configuration driven via TestNG suite parameters (@Parameters) and environment variables.
- **Data Provider & Matrix Execution:** Cartesian product BrowserDataProvider mapping every CSV record against target browser drivers.
- **Lifecycle:** Method-level driver instantiation and explicit teardown with home page navigation resets.
- **Failure Diagnostics:** TestFailureListener capturing URL, page title, failure taxonomy (Locator vs. Application assertion), and PNG screenshots upon failure

Project Structure: -

```
selenium-automation-framework/
├── pom.xml
├── external-test-data/
│   └── LoginTest.csv
└── src/
    ├── main/
    │   └── java/
    │       └── com/
    │           └── automation/
    │               ├── config/
    │               │   └── ConfigManager.java
    │               ├── driver/
    │               │   ├── DriverFactory.java
    │               │   └── DriverManager.java
    │               ├── pages/
    │               │   ├── BasePage.java
    │               │   ├── HomePage.java
```

```
│           │   └── LoginPage.java
│           └── utils/
│               ├── BrowserUtils.java
│               ├── ScreenshotUtils.java
│               └── WaitUtils.java
└── test/
    ├── java/
    │   └── com/
    │       └── automation/
    │           ├── dataproviders/
    │           │   ├── BrowserDataProvider.java
    │           │   └── CsvDataProvider.java
    │           ├── listeners/
    │           │   └── TestFailureListener.java
    │           └── tests/
    │               └── LoginTest.java
    └── resources/
        └── suites/
            ├── smoke.xml
            ├── sanity.xml
            └── regression.xml
```

Complete Aligned Codebase

1. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.automation</groupId>
```

```
<artifactId>selenium-automation-framework</artifactId>
<version>1.0.0</version>

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.release>21</maven.compiler.release>
  <selenium.version>4.46.0</selenium.version>
  <testng.version>7.12.0</testng.version>
  <opencsv.version>5.12.0</opencsv.version>
  <maven.compiler.plugin.version>3.15.0</maven.compiler.plugin.version>
  <maven.surefire.version>3.5.3</maven.surefire.version>
</properties>

<dependencies>
  <!-- Selenium -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>${selenium.version}</version>
  </dependency>
  <!-- TestNG -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>${testng.version}</version>
    <scope>test</scope>
  </dependency>
  <!-- CSV parser -->
  <dependency>
    <groupId>com.opencsv</groupId>
    <artifactId>opencsv</artifactId>
```

```
        <version>${opencsv.version}</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- Java compiler plugin with release 21 -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>${maven.compiler.plugin.version}</version>
        <configuration>
          <release>21</release>
        </configuration>
      </plugin>
      <!-- TestNG execution plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>${maven.surefire.version}</version>
      </plugin>
    </plugins>
  </build>
</project>
```

2. Configuration Management

src/main/java/com/automation/config/ConfigManager.java

```
package com.automation.config;

/**
 * Validates and holds parameters passed from TestNG XML suite execution.
 */
```

```
public final class ConfigManager {
    private static String browser;
    private static String baseUrl;

    private ConfigManager() {
        // Prevent object creation
    }

    public static void initialize(String browser, String baseUrl) {
        if (browser == null || browser.isBlank()) {
            throw new IllegalArgumentException("Browser parameter is mandatory."); //[cite: 1]
        }
        if (baseUrl == null || baseUrl.isBlank()) {
            throw new IllegalArgumentException("Base URL parameter is mandatory."); //[cite: 1]
        }
        ConfigManager.browser = browser.toLowerCase(); //[cite: 1]
        ConfigManager.baseUrl = baseUrl; //[cite: 1]
    }

    public static String getBrowser() {
        return browser; //[cite: 1]
    }

    public static String getBaseUrl() {
        return baseUrl; //[cite: 1]
    }
}
```

3. Driver Management & Factory Pattern

src/main/java/com/automation/driver/DriverManager.java

```
package com.automation.driver;
```

```
import org.openqa.selenium.WebDriver;

/**
 * Ensures complete thread isolation by assigning a distinct WebDriver instance per execution thread[cite: 1].
 */
public final class DriverManager {

    private DriverManager() {
    }

    private static final ThreadLocal<WebDriver> DRIVER = new ThreadLocal<>(); //[cite: 1]

    public static void setDriver(WebDriver driver) {
        DRIVER.set(driver); //[cite: 1]
    }

    public static WebDriver getDriver() {
        WebDriver driver = DRIVER.get(); //[cite: 1]
        if (driver == null) {
            throw new IllegalStateException("WebDriver has not been initialized for this thread."); //[cite: 1]
        }
        return driver; //[cite: 1]
    }

    public static void quitDriver() {
        WebDriver driver = DRIVER.get(); //[cite: 1]
        if (driver != null) {
            try {
                driver.quit(); //[cite: 1]
```

```
        } finally {
            DRIVER.remove(); // Prevents ThreadLocal memory leaks[cite: 1]
        }
    }
  }
}
```

src/main/java/com/automation/driver/DriverFactory.java

```
package com.automation.driver;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.safari.SafariDriver;

/**
 * Creates browser driver instances using native Selenium Manager discovery[cite: 1].
 */
public final class DriverFactory {

  private DriverFactory() {
  }

  public static WebDriver createDriver(String browser) {
    if (browser == null || browser.isBlank()) {
      throw new IllegalArgumentException("Browser cannot be null or empty."); //[cite: 1]
    }
    return switch (browser.toLowerCase()) {
      case "chrome" -> new ChromeDriver(); //[cite: 1]
      case "firefox" -> new FirefoxDriver(); //[cite: 1]
      case "edge" -> new EdgeDriver(); //[cite: 1]
```

```
        case "safari" -> new SafariDriver(); // Requires macOS execution host[cite: 1]
        default -> throw new IllegalArgumentException("Unsupported browser: " + browser); //[cite: 1]
      };
    }
}
```

4. Framework Utilities

src/main/java/com/automation/utils/WaitUtils.java

```
package com.automation.utils;

import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

/**
 * Handles explicit element synchronization without hardcoded sleep delays[cite: 1].
 */
public final class WaitUtils {

    private static final Duration DEFAULT_TIMEOUT = Duration.ofSeconds(15); //[cite: 1]

    private WaitUtils() {
    }

    public static WebElement waitForVisible(WebDriver driver, By locator) {
        return new WebDriverWait(driver, DEFAULT_TIMEOUT)
                .until(ExpectedConditions.visibilityOfElementLocated(locator)); //[cite: 1]
    }
```

```
    public static WebElement waitForClickable(WebDriver driver, By locator) {
        return new WebDriverWait(driver, DEFAULT_TIMEOUT)
                .until(ExpectedConditions.elementToBeClickable(locator)); //[cite: 1]
    }

    public static boolean waitForUrlContains(WebDriver driver, String value) {
        return new WebDriverWait(driver, DEFAULT_TIMEOUT)
                .until(ExpectedConditions.urlContains(value)); //[cite: 1]
    }
}
```

src/main/java/com/automation/utils/BrowserUtils.java

```
package com.automation.utils;

import org.openqa.selenium.WebDriver;

public final class BrowserUtils {

    private BrowserUtils() {
    }

    public static void maximize(WebDriver driver) {
        driver.manage().window().maximize(); //[cite: 1]
    }

    public static void navigateTo(WebDriver driver, String url) {
        driver.get(url); //[cite: 1]
    }

    public static void navigateHome(WebDriver driver, String homeUrl) {
        driver.navigate().to(homeUrl); //[cite: 1]
    }
```

```
}
```

src/main/java/com/automation/utils/ScreenshotUtils.java

```
package com.automation.utils;

import java.io.File;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;

public final class ScreenshotUtils {

  private ScreenshotUtils() {
  }

  public static Path capture(WebDriver driver, String testName) {
    try {
      Path directory = Path.of("test-output", "screenshots"); //[cite: 1]
      Files.createDirectories(directory); //[cite: 1]

      String fileName = testName + "_" + System.currentTimeMillis() + ".png"; //[cite: 1]
      Path destination = directory.resolve(fileName); //[cite: 1]

      File source = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE); //[cite: 1]
      Files.copy(source.toPath(), destination, StandardCopyOption.REPLACE_EXISTING); //[cite: 1]

      return destination; //[cite: 1]
    } catch (Exception e) {
      System.err.println("Unable to capture screenshot: " + e.getMessage()); //[cite: 1]
```

```
        return null; //[cite: 1]
      }
    }
}
```

5. Data Providers & CSV Ingestion

src/test/java/com/automation/dataproviders/CsvDataProvider.java

```
package com.automation.dataproviders;

import com.opencsv.CSVReader;
import com.opencsv.exceptions.CsvValidationException;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public final class CsvDataProvider {

  private CsvDataProvider() {
  }

  public static List<String[]> readCsv(String filePath) {
    List<String[]> rows = new ArrayList<>(); //[cite: 1]
    try (CSVReader reader = new CSVReader(new FileReader(filePath))) { //[cite: 1]
      String[] headers = reader.readNext(); // Skip header row[cite: 1]
      if (headers == null) {
        throw new IllegalArgumentException("CSV file is empty: " + filePath); //[cite: 1]
      }
      String[] row;
      while ((row = reader.readNext()) != null) {
        rows.add(row); //[cite: 1]
      }
```

```
    } catch (IOException | CsvValidationException e) {
      throw new RuntimeException("Unable to read CSV: " + filePath, e); //[cite: 1]
    }
    return rows; //[cite: 1]
  }
}
```

src/test/java/com/automation/dataproviders/BrowserDataProvider.java

```
package com.automation.dataproviders;

import org.testng.annotations.DataProvider;
import java.util.ArrayList;
import java.util.List;

/**
 * Dynamically constructs a Cartesian Product mapping CSV records against browser types[cite: 1].
 */
public final class BrowserDataProvider {

  private BrowserDataProvider() {
  }

  @DataProvider(name = "browserAndCsvData", parallel = true) //[cite: 1]
  public static Object[][] browserAndCsvData() {
    String csvPath = System.getenv("AUTOMATION_CSV_PATH"); //[cite: 1]
    if (csvPath == null || csvPath.isBlank()) {
      throw new IllegalStateException("AUTOMATION_CSV_PATH environment variable is mandatory.");
//[cite: 1]
    }

    List<String[]> csvRows = CsvDataProvider.readCsv(csvPath); //[cite: 1]
    String[] browsers = {"chrome", "firefox", "edge", "safari"}; // Matrix browsers[cite: 1]
```

```
    List<Object[]> executionData = new ArrayList<>(); //[cite: 1]
    for (String[] row : csvRows) {
      for (String browser : browsers) {
        executionData.add(new Object[]{browser, row}); //[cite: 1]
      }
    }
    return executionData.toArray(new Object[0][]); //[cite: 1]
  }
}
```

6. Page Object Layer (Direct By Locators)

src/main/java/com/automation/pages/BasePage.java

```
package com.automation.pages;

import org.openqa.selenium.WebDriver;

public abstract class BasePage {
  protected final WebDriver driver; //[cite: 1]

  protected BasePage(WebDriver driver) {
    this.driver = driver; //[cite: 1]
  }
}
```

src/main/java/com/automation/pages/LoginPage.java

```
package com.automation.pages;

import com.automation.utils.WaitUtils;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage extends BasePage {
```

```
    // Direct By locators inside Page Objects to avoid stale proxies[cite: 1]
    private final By username = By.id("username"); //[cite: 1]
    private final By password = By.id("password"); //[cite: 1]
    private final By loginButton = By.id("loginButton"); //[cite: 1]

    public LoginPage(WebDriver driver) {
        super(driver); //[cite: 1]
    }

    public void enterUsername(String value) {
        WaitUtils.waitForVisible(driver, username).sendKeys(value); //[cite: 1]
    }

    public void enterPassword(String value) {
        WaitUtils.waitForVisible(driver, password).sendKeys(value); //[cite: 1]
    }

    public void clickLogin() {
        WaitUtils.waitForClickable(driver, loginButton).click(); //[cite: 1]
    }

    public HomePage login(String usernameValue, String passwordValue) {
        enterUsername(usernameValue); //[cite: 1]
        enterPassword(passwordValue); //[cite: 1]
        clickLogin(); //[cite: 1]
        return new HomePage(driver); //[cite: 1]
    }
}
```

src/main/java/com/automation/pages/HomePage.java

```
package com.automation.pages;
```

```
import com.automation.utils.WaitUtils;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class HomePage extends BasePage {

  private final By homeHeader = By.id("homeHeader"); //[cite: 1]

  public HomePage(WebDriver driver) {
    super(driver); //[cite: 1]
  }

  public boolean isDisplayed() {
    return WaitUtils.waitForVisible(driver, homeHeader).isDisplayed(); //[cite: 1]
  }
}
```

7. Failure Listener & Diagnostics

src/test/java/com/automation/listeners/TestFailureListener.java

```
package com.automation.listeners;

import com.automation.driver.DriverManager;
import com.automation.utils.ScreenshotUtils;
import org.openqa.selenium.WebDriver;
import org.testng.ITestListener;
import org.testng.ITestResult;

public class TestFailureListener implements ITestListener {

  @Override
```

```
public void onTestFailure(ITestResult result) {
    try {
        WebDriver driver = DriverManager.getDriver(); //[cite: 1]
        String testName = result.getMethod().getMethodName(); //[cite: 1]

        System.err.println("TEST FAILURE"); //[cite: 1]
        System.err.println("Test: " + testName); //[cite: 1]
        System.err.println("URL: " + driver.getCurrentUrl()); //[cite: 1]
        System.err.println("Title: " + driver.getTitle()); //[cite: 1]
        System.err.println("Failure Type: " + classifyFailure(result.getThrowable())); //[cite: 1]

        ScreenshotUtils.capture(driver, testName); //[cite: 1]
    } catch (Exception e) {
        System.err.println("Failure listener error: " + e.getMessage()); //[cite: 1]
    }
}

private String classifyFailure(Throwable throwable) {
    if (throwable == null) {
        return "UNKNOWN"; //[cite: 1]
    }
    String name = throwable.getClass().getSimpleName(); //[cite: 1]
    if (name.contains("Timeout")) {
        return "AUTOMATION/SYNCHRONIZATION"; //[cite: 1]
    }
    if (name.contains("NoSuchElement")) {
        return "AUTOMATION/LOCATOR"; //[cite: 1]
    }
    if (name.contains("WebDriver")) {
        return "AUTOMATION/WEBDRIVER"; //[cite: 1]
    }
```

```
    if (name.contains("Assertion")) {
      return "APPLICATION/TEST_VALIDATION"; //[cite: 1]
    }
    return "APPLICATION_OR_AUTOMATION"; //[cite: 1]
  }
}
```

8. Concrete Test Class Implementation

src/test/java/com/automation/tests/LoginTest.java

```
package com.automation.tests;

import com.automation.config.ConfigManager;
import com.automation.driver.DriverFactory;
import com.automation.driver.DriverManager;
import com.automation.listeners.TestFailureListener;
import com.automation.pages.HomePage;
import com.automation.pages.LoginPage;
import com.automation.utils.BrowserUtils;
import org.openqa.selenium.WebDriver;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Listeners;
import org.testng.annotations.Parameters;
import org.testng.annotations.Test;

@Listeners(TestFailureListener.class) //[cite: 1]
public class LoginTest {

  @BeforeMethod
  @Parameters({"browser", "baseUrl"}) // Configuration injected via TestNG XML[cite: 1]
```

```
public void setUp(String browser, String baseUrl) {
   ConfigManager.initialize(browser, baseUrl); //[cite: 1]
   WebDriver driver = DriverFactory.createDriver(browser); //[cite: 1]
   DriverManager.setDriver(driver); //[cite: 1]
   BrowserUtils.navigateTo(driver, baseUrl); //[cite: 1]
}

@Test(priority = 1, groups = {"smoke", "sanity", "regression"}) //[cite: 1]
public void verifyValidLogin() {
   WebDriver driver = DriverManager.getDriver(); //[cite: 1]

   // Secure environment credential fetching[cite: 1]
   String username = System.getenv("AUTOMATION_USERNAME"); //[cite: 1]
   String password = System.getenv("AUTOMATION_PASSWORD"); //[cite: 1]

   Assert.assertNotNull(username, "AUTOMATION_USERNAME is missing."); //[cite: 1]
   Assert.assertNotNull(password, "AUTOMATION_PASSWORD is missing."); //[cite: 1]

   LoginPage loginPage = new LoginPage(driver); //[cite: 1]
   HomePage homePage = loginPage.login(username, password); //[cite: 1]

   Assert.assertTrue(homePage.isDisplayed(), "Home page should be displayed after login."); //[cite: 1]
}

@AfterMethod
public void tearDown() {
   try {
      WebDriver driver = DriverManager.getDriver(); //[cite: 1]
      try {
         // Navigate home to reset state before terminating session[cite: 1]
         BrowserUtils.navigateHome(driver, ConfigManager.getBaseUrl()); //[cite: 1]
```

```
        } catch (Exception e) {
            System.err.println("Unable to navigate home during cleanup: " + e.getMessage()); //[cite: 1]
        }
    } finally {
        DriverManager.quitDriver(); // Always clear ThreadLocal driver session[cite: 1]
    }
  }
}
```

9. Test Suite XML Definitions

src/test/resources/suites/smoke.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Smoke Suite" parallel="methods" thread-count="4">
  <test name="Smoke Tests">
    <parameter name="browser" value="chrome"/>
    <parameter name="baseUrl" value="https://your-application-url.com"/>
    <groups>
      <run>
        <include name="smoke"/>
      </run>
    </groups>
    <classes>
      <class name="com.automation.tests.LoginTest"/>
    </classes>
  </test>
</suite>
```

src/test/resources/suites/regression.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Regression Suite" parallel="methods" thread-count="4">
```

```
  <test name="Regression Tests">
    <parameter name="browser" value="chrome"/>
    <parameter name="baseUrl" value="https://your-application-url.com"/>
    <groups>
      <run>
        <include name="regression"/>
      </run>
    </groups>
    <classes>
      <class name="com.automation.tests.LoginTest"/>
    </classes>
  </test>
</suite>
```

10. Sample Test Data Schema

external-test-data/LoginTest.csv

```
TestCaseId,Username,Password,ExpectedResult
TC_LOGIN_001,validuser,${AUTOMATION_PASSWORD},SUCCESS
TC_LOGIN_002,invaliduser,${AUTOMATION_PASSWORD},FAILURE
```

Execution Environment Setup & Commands

Set your required environment variables prior to running test execution[cite: 1]:

```
# Windows PowerShell example
$env:AUTOMATION_USERNAME="student"
$env:AUTOMATION_PASSWORD="Password123"
$env:AUTOMATION_CSV_PATH="external-test-data/LoginTest.csv"
```

To run individual execution suites using Maven CLI[cite: 1]:

```
# Execute Smoke Suite
mvn clean test -Dsurefire.suiteXmlFiles=src/test/resources/suites/smoke.xml

# Execute Sanity Suite
```

```
mvn clean test -Dsurefire.suiteXmlFiles=src/test/resources/suites/sanity.xml

# Execute Regression Suite
mvn clean test -Dsurefire.suiteXmlFiles=src/test/resources/suites/regression.xml
```